Método de Bisección

Javier Alexander López Contreras
Facultad de Ciencias Físico Matemáticas (FCFM), Universidad Autónoma de Coahuila, Campus Campo redondo

Recibido el 18/02/2026

En este documento se analizará el método de bisección para encontrar la aproximación de las raíces de una función, utilizando un programa escrito en lenguaje python en el interprete Visual Studio Code.

1. INTRODUCCIÓN

El método de bisección es uno de los mejores algoritmos para la aproximación de funciones continuas. Este método busca resolver ecuaciones no lineales mediante la división sistemática de intervalos del **Teorema de bolzano**, el cual establece que si una función *f(x)* es continua en un intervalo cerrado *[a,b]* y los valores de la funcion en los extremos tienen signos opuestos ($f(a)\cdot f(b) < 0$), entonces debe existir al menos un valor c dentro de dicho intervalo tal que $f(c) = 0$.

2. ALGORITMO Y EXPLICACIÓN DEL MÉTODO DE BISECCIÓN

El fundamento del método es identificar un intervalo $[a, b]$ donde se cumpla la condición del Teorema de Bolzano.

- Se definen los límites iniciales a y b tales que la función evaluada en ellos tenga signos opuestos.
- Se calcula el punto medio p del intervalo actual:

$$c = a + \frac{b-a}{2} \tag{1}$$

- Se evalúa el signo de $f(c)$
  - Si $f(a)\cdot f(c) < 0$, la raíz se encuentra en el nuevo intervalo $[a, c]$
  - Si $f(a) \cdot f(c) > 0$, la raíz se encuentra en el intervalo $[c, b]$
- El proceso se detiene cuando se alcanza una tolerancia de error prefijada. Para cuantificar la precisión el programa calcula:
  - **Error absoluto**(E_a): Magnitud de la diferencia entre el valor actual y el anterior:

$$E_a = |p_i - p_{i-1}| \tag{2}$$

  - **Error relativo** (E_r): Relación entre el error absoluto y el valor actual:

$$E_r = \left|\frac{p_i - p_{i-1}}{pi}\right| \tag{3}$$

3. ANÁLISIS DEL CÓDIGO

El programa en python desarrollado con **tkinter** permite el ingreso de funciones personalizadas.

3.1. Configuración de la interfaz y entrada de datos.

En esta sección se define la ventana principal y la captura de la función matemática a analizar. Un aspecto crítico es el uso de **eval()**, que permite la interpretación dinámica de la ecuación ingresada por el usuario.

```python

ventana = tk.Tk()
ventana.title('Programa para encontrar las ra ces de un polinomio')

def f(x):
    return eval(funcion.get())

funcion = tk.Entry(ventana, width=50)
funcion.pack()
```

Listing 1: Configuración inicial y captura de la función.

3.2. Gráfica de la función

Para que el usuario sea capaz de saber que intervalo tomar, se implementó un botón que muestra la gráfica de la función ingresada.

```python

def grafica():
    x = np.linspace(-10, 10, 400)
    y = f(x)
    plt.plot(x,y, label='f(x)')
    plt.axhline(0, color='blue', lw=0.5, ls='')
    plt.axvline(0, color='blue', lw=0.5, ls='')
    plt.title(f'Gr fica de f(x) = {funcion.get()}')
    plt.xlabel('x')
    plt.ylabel('f(x)')
    plt.grid()
    plt.legend()
    plt.show()

boton_grafica = tk.Button(ventana, text='Mostrar gr fica', command=lambda: [limpiar_grafica(), grafica()])
boton_grafica.pack()
```

Listing 2: Gráfica de la función y boton para mostrarla

3.3. Operaciónes para realizar el método y los errores absoluto y relativo.

En esta sección se traduce el fundamento matemático del método de bisección a un proceso iterativo.

- **Cálculo del punto medio:** Para determinar la aproximación de la raíz en cada iteración, el código utiliza la fórmula de incremento, ya que previene errores de desbordamiento y mantiene la precisión dentro de los límites de la aritmética de punto flotante de la computadora. Además, aquí se evalúan los puntos para saber cual será el nuevo intervalo.

```
c = a + (b - a) / 2
if f(a) * f(c) < 0:
    b = c
else:
    a = c
```

Listing 3: Operación del punto medio y lógica de decisión.

- **Cuantificación del error:** El programa no solo entrega la raíz aproximada, sino que calcula la precisión de la ultima iteración. Esto es fundamental debido a que los números reales a veces no pueden ser representados exactamente en memoria, generando "huecos"de precisión

  - **Error absoluto (E_a):**En el código se define como la distancia entre los límites actuales del intervalo $|b - a|$. Representa la incertidumbre máxima de la aproximación actual.
  - **Error Relativo (E_r):** Se calcula dividiendo el error absoluto entre el valor absoluto del punto medio actual. Se usa para entender la escala del error respecto a la magnitud del resultado, similar a la perdida de exactitud cuando se trabaja con fracciones.

```
error_absoluto = abs(b - a)
error_relativo = error_absoluto / abs(c) if c != 0
    else float('inf')
```

Listing 4: Cálculo de errores según la implementación del programa.

4. RESULTADOS Y ANÁLISIS

En esta etapa se evaluó el desempeño del software frente a funciones con comportamientos específicos que ponen a prueba la precisión de la aritmética de punto flotante.

4.1. Raíces múltiples en un mismo intervalo

Al analizar funciones como polinomios de tercer grado (raíces cúbicas), se presenta un fenómeno interesante, una función puede tener tres raíces reales, pero si el intervalo $[a, b]$ las contiene todas, el método de bisección entregará solo **una raíz**.

Esto sucede porque el algoritmo no busca las soluciones simultáneamente, sino que se rige estrictamente por el cambio de signo de Bolzano. Al dividir el intervalo, el programa descarta la mitad donde no se detecta el cambio de signo, atrapando solo una de las 3 iteraciones. Si los límites son exactos, el programa entrega un valor con error cero, pero ignorará las otras dos soluciones.

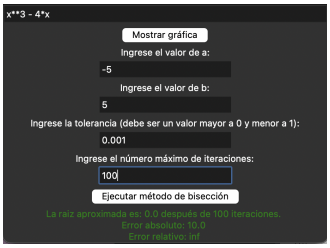

Figura 1: Prueba con funciones cúbicas

4.2. Funciones asintóticas y el problema del cero computacional

Un caso crítico ocurre cuando una función se aproxima infinitamente al eje x sin llegar a tocarlo nunca .Matemáticamente, la raíz no existe, pero constitucionalmente el programa enfrenta un riesgo de **desbordamiento o iteraciones infinitas**

- **Iteraciones infinitas:**Si la función tiende a cero pero nunca cruza el eje, la condición $f(a) \cdot f(b) < 0$ puede no cumplirse nunca o mantenerse en un estado donde el error absoluto disminuye tan lentamente que el programa agota el **max**$_i tersinencontrarunasolucin.$

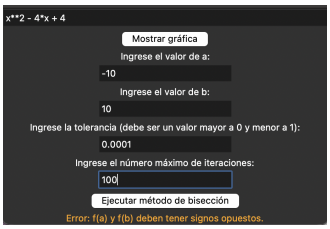

Figura 2: Prueba con función que se aproxima al eje x

4.3. Análisis de errores de salida:

En las pruebas realizadas, también se observó que mientras el error absoluto cuantifica la cercanía entre los límites, el error relativo es el que realmente nos indica si la aproximación es satisfactoria. Si el punto medio c es un número muy pequeño, el error relativo tiende a crecer drásticamente, lo que obliga a interpretas los resultados no como verdades absolutas, sino como aproximaciónes de un entorno de cálculo aproximado.

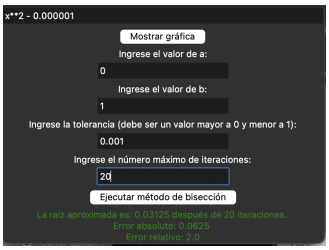

Figura 3: Diferencia en errores de salida

5. CONCLUSIONES

Tras la implementación y experimentación con el método de bisección, se puede concluir que su eficiencia no reside en la velocidad de convergencia, si no en su **absoluta fiabilidad** bajo las condiciones del Teorema de Bolzano. A diferencia de otros algoritmos, la bisección garantiza el hallazgo de una aproximación siempre que el intervalo inicial esté correctamente delimitado. También se observo que ante la presencia de múltiples raíces en un mismo intervalo, el método converge solo en una solución basándose en las divisiones sucesivas, lo que nos obliga a realizar un estudio previo de la función. Al no detectar un cambio de signo, el algoritmo evita procesar datos donde no existe una solución real, protegiendo de posibles infinitas iteraciones.

También revela que el error absoluto puede ser engañoso, ya que para raíces muy pequeñas, el error relativo es el único indicador para validar si la aproximación es satisfactoria o si tiene ruido de precisión debido a la aritmética del punto flotante de la computadora.

Para finalizar, este método parece una herramienta indispensable en la ingeniería. Aunque la naturaleza finita de la computadora impone límites de exactitud, el control de la tolerancia permite obtener resultados más confiables y reproducibles en un entorno de cálculo aproximado.

6. REFERENCIAS

[1] Nieves, A. y Domínguez, F. C. (2014). *Métodos numéricos aplicados a la ingeniería* (4a ed.). México: Grupo Editorial Patria.

[2] Chapra, S. C. y Canale, R. P. (2015). *Métodos numéricos para ingenieros* (7a ed.). México: McGraw-Hill.

[3] Moler, C. B. (2004). *Numerical Computing with MATLAB*. SIAM.

[4] Python Software Foundation. (2026). *The Python Tutorial: Floating Point Arithmetic, Issues and Limitations*. Recuperado de https://docs.python.org/3/tutorial/floatingpoint.html